Target Architecture

Principles

1. **Ideas are the atomic citation unit.** Claims paired with verbatim anchor quotes, extracted per document. Never re-written, never canonicalized across documents. Every downstream artifact cites ideas.
2. **Topics are the article unit.** One article per topic. Topics are thematic abstractions, not entities. A single real-world person, event, or work may appear in many topics' articles; a topic may span many real-world entities.
3. **Ideas ↔ topics is many-to-many.** Derived from map/reduce provenance, not from clustering or retrieval.
4. **No intermediate concept/entity layer.** Entity identity is not a first-class structural concern. It's handled implicitly inside render when the LLM writes about a person/event across its contributing idea fragments.
5. **Batch-agnosticism is a structural property (within a cache snapshot).** Given a cache C recording the LLM's outputs for corpus state S, any incremental compile drawing from a subset of C produces the same canonical topics as a full compile over C. Convergence across independent LLM passes is not claimed — nor achievable — because extract/map/reduce/render are stochastic. Identity stability across compiles comes from slug continuity for topics and from the cache returning recorded outputs for everything below the LLM surfaces. UUIDs themselves are opaque identifiers; they carry no semantic content.
6. **Stochastic surfaces are bounded and named.** Exactly four LLM-call types: extract, map, reduce, render. Everything else is mechanical.
7. **Raw markdown is immutable.** User feedback and lint findings re-enter the pipeline as new source documents.
8. **Per-brain configuration shapes the pipeline.** The `kind` taxonomy and the thematic-schema hint to the reducer are declared at brain creation. Defaults exist; overrides are cheap.

Per-brain configuration

Declared at brain creation, stored alongside the brain:

```
kinds:
  - person
  - event
  - organization
  - movement
  - concept
  # ... whatever taxonomy fits the domain
thematic_hint: |
  Prefer topics shaped like events, movements, historical
  conjunctures, and intellectual debates. Avoid biography-first
  framings.
```

`kinds` is consumed by extract's prompt to constrain idea-level classification. `thematic_hint` is prepended to reduce's prompt. Both have defaults.

Data model

table	authoritative	purpose
`documents`	Postgres	one row per ingested raw file; title, doc_metadata, precis
`source_cards` (`.compile/<brain>/source_cards.jsonl`)	JSONL on disk	per-document extraction output: ideas, anchors
`idea_embeddings`	Postgres (pgvector)	one vector per idea; used for partition
`topics`	Postgres	canonical theme registry: slug, title, description, lifecycle
`topic_membership`	Postgres	derived: `(topic_id, idea_id)` edges
`topic_links`	Postgres	derived from reduce's validated link_targets
`topic_related`	Postgres	derived: shared-idea relatedness for sidebar UI
`backlinks`	Postgres	derived: reverse of `topic_links`
`chunks` (raw + wiki)	Postgres	paragraph/heading-level anchors + embeddings for agent RAG

The `concepts` table and its associated code (`distiller.py` clustering, `concept_repository.py`) are retired.

Ideas — schema (from extract)

```
idea_id           uuid7 (minted at extract time; opaque)
document_id       uuid
kind              one of brain's configured kinds (fallback: other)
label             short canonical name as it appears in this doc
description       one-sentence description of what this is, as treated
anchors           list of {anchor_id, claim, quote}
```

Idea rows are rewritten on every extract re-run for a document: delete-then-insert keyed on `document_id`. Cache hits on `sha256(doc_content + prompt_version + kinds_config + extract_model)` short-circuit extract entirely and return the stored `source_card` verbatim, so `idea_ids` stay stable across incremental compiles without any derivation trick. On cache miss the LLM re-draws and fresh ids are minted — that's expected, not a regression.

`kind` lives at the idea layer because ideas describe specific factual things (a person, an event) and kind is useful for idea-level filtering and as a partition hint. Topics are thematic abstractions that span multiple kinds by design, so they have no `kind` field.

Topics — schema (from reduce)

```
topic_id            uuid7 (minted fresh, stabilized via slug continuit
brain_id            uuid
slug                kebab-case, unique within brain
```

```
title                 human-readable theme title
description           2-3 sentences describing what this topic covers
article_status        no_article | rendered | needs_revision | archived
rendered_from_hash    hash of the idea set at last successful render
compiled_from_hash    hash of the current idea set (dirty if != rendered
supersedes            topic_id that this one replaced (nullable)
superseded_by         topic_id that replaced this one (nullable)
```

Doc-level metadata threaded into ideas

When ideas are prepared for map's prompt, each idea is rendered with provenance:

```
[kind] label: description
  ← from {title} ({genre}, {date}) by {author}; interlocutors: {...}; t
  ← precis: {2-3 sentences on what the doc is doing}
```

This gives the map LLM whole-doc context per fragment without a separate per-doc plan step.

Pipeline

The compilation pipeline has seven named phases: `ingest → extract → abstract → derive → render → verify → publish`. "Compile" is the umbrella term for the whole pipeline.

Phase 0 — ingest (mechanical)

Per document:

- Store raw markdown in `raw/<source_type>/<slug>.md` (immutable)
- Paragraph-level chunking with anchor IDs; embed chunks; write to `chunks` table
- No LLM calls

Raw-doc chunking gives the agent RAG access to primary sources.

Phase 1 — extract (LLM, one call per document)

Per document (cache key: `sha256(doc_content)`):

- Single LLM call reads raw doc + source_type + brain's kinds config
- Output: `title`, `doc_metadata` (genre, tags, tradition, interlocutors), `precis` (2-3 sentences), `ideas[]` (claims + anchors), `anchors[]` (verbatim quotes)
- Write to `source_cards.jsonl`; write `documents` row with title + metadata + precis
- Embed ideas (`qwen3-embedding-8b` on `label + description`), store in `idea_embeddings`

A doc-generated title is critical because filenames are often uninformative (e.g., `04x.md`, `M00.md`).

Model: DeepSeek v3.2 (→ V4 when live). Cheap, volume-heavy.

Phase 2 — abstract (mixed)

Five sub-steps, only two of which are LLM calls.

2a. Partition (mechanical). Cache key: `sha256(sorted idea_ids)`.

- Load all idea embeddings
- Seeded k-means with `k = total_trimmed_tokens / target_chunk_tokens` (target ~100K tokens per chunk)
- Size-rebalance pass: subdivide chunks exceeding token budget, merge chunks below a floor
- Output: `chunks[]`, each a list of `idea_ids`

Fully deterministic for a given embedding set.

2b. Map (LLM, one call per chunk). Cache key: `sha256(sorted idea_ids in chunk)`.

- Render each idea with doc-level provenance (title, precis, metadata)
- Prompt asks for 10–30 thematic topics covering this subset; each topic has `slug`, `title`, `description`, `subsumed_idea_ids[]`
- Temperature 0.3, structured JSON output

Model: DeepSeek v3.2 (→ V4). ~30 calls at 10K-doc scale.

2c. Pre-merge (mechanical). Collapse obvious duplicates from map outputs using exact-match signals only:

- Identical slugs merge
- Identical titles merge
- Jaccard similarity of `subsumed_idea_ids` > 0.8 merge

Union `subsumed_idea_ids` on merge. No cosine/embedding signal — exact matches are precise; subtler merges are left to the reducer LLM, which has global view.

2d. Reduce (LLM, one call). Cache key: `sha256(sorted local topic ids + their hashes)`.

Prompt:

- Brain's `thematic_hint` prepended
- Reads all pre-merged local topics at once
- Produces the **canonical topic registry** for this compile
- Also produces `link_targets[]` per canonical topic

Structured output:

```
{
  "canonical_topics": [
    {
      "slug": "kebab-case",
      "title": "...",
      "description": "2-3 sentences",
      "merged_local_topic_ids": ["t17", "t34", "..."],
      "link_targets": ["other-canonical-slug-1", "other-canonical-slug-
```

```
    }
  ]
}
```

Model: **Qwen 3.6 Plus** (primary). Fallback to Claude Sonnet 4.6 if quality insufficient; Claude Opus 4.7 if Sonnet is insufficient.

Scale escape (in order of preference):

1. Baseline reduce (fits in budget): one LLM call sees everything.
2. Pre-merge + reduce: exact-match compression, reducer still sees all survivors.
3. Aggressive pre-merge + reduce: looser Jaccard threshold (~0.6), normalized title matching (case/punctuation-insensitive). Same reducer view.
4. Hierarchical reduce: cluster local topics by description embedding, reduce each cluster, then final reduce over sub-canonical outputs. Last resort — top-level reducer loses global view.

Validation (mechanical, part of abstract):

- Intersect each topic's `link_targets` with the set of emitted `slugs`; drop non-matches (closes the hallucinated-link error surface)
- Reconcile topic IDs via slug continuity: for any canonical slug already in `topics`, reuse the existing `topic_id`; otherwise mint UUID7
- M7 archive flow: any existing topic whose slug is NOT in the new canonical set gets `article_status=archived`, moved to `archive/<topic_id>/<old_slug>.md` with `superseded_by` pointer

End of phase 2: the `topics` registry is updated and the merge structure is in hand.

Phase 3 — derive (mechanical)

Build relational tables from reduce's validated output. These tables reflect **topic-level intent**, not what ends up in article prose.

- `topic_membership`: for each canonical topic, union the `subsumed_idea_ids` from all its merged local topics
- `topic_links`: one edge per `(source_topic_id, target_topic_id)` from validated `link_targets`. Represents intended citations.
- `topic_related` (for sidebar UI): for each topic pair, count of shared ideas and Jaccard over their idea sets; stored sorted per topic
- `compiled_from_hash` on each topic: hash of its `topic_membership` + `title` + `description`

`backlinks` is NOT built here — it belongs to article-level reality, not topic-level intent, and is constructed in verify.

Phase 4 — render (LLM, one call per topic)

Per topic (cache key: `sha256(topic_id + topic_membership_hash + link_targets)`):

- Renderer receives: topic (title, description), ideas (full records with anchors), `link_targets` (authoritative list)
- Prompt instructs: write the article body, use `[title](wiki/<slug>.md)` markdown

links when referencing any of the `link_targets` by concept, cite anchors as `[^n]` footnotes
- Output: markdown article body

Model: **Qwen 3.6 Plus** (primary), upgrade to Sonnet 4.6 if prose quality matters.

Write to `wiki/<slug>.md` with frontmatter (`topic_id`, `title`, `description`). Set `rendered_from_hash = compiled_from_hash`, `article_status = rendered`.

Re-chunk rendered articles and update wiki-article chunks in `chunks` (retains existing `rebuild_index` behavior).

Phase 5 — verify (mechanical)

Verify operates on article-level reality — what rendered prose actually contains.

- Walk rendered articles, extract `[title](wiki/<slug>.md)` citations from each body
- Build `backlinks` table from actual article citations: `(target_topic_id, source_topic_id, source_article_path)` reflecting real article-to-article links
- Log unresolved links (citation points to a nonexistent slug) as lint signals
- Log intended-but-unmentioned discrepancies (link_target from `topic_links` that doesn't appear in the article body) as quality signals

The split: `topic_links` is topic-level intent (from reduce); `backlinks` is article-level reality (from rendered prose). They can diverge, and verify surfaces the divergence.

Phase 6 — publish (mechanical)

- Generate `wiki/index.md`: topic list with title + description
- Generate `raw/index.md`: doc list with title + precis + doc_metadata
- Append compile entry to `.compile/<brain>/log.md`: timestamp, topic counts, diff vs. prior compile

Incremental compilation

Every LLM phase is cached by content hash. Invalidation propagates:

```
doc changes
  → extract re-runs for that doc
  → source_cards.jsonl updated
  → idea_embeddings updated for changed ideas
  → partition re-runs (idea_set_hash changed)
  → chunks with changed composition → map re-runs
  → map outputs hash changed → reduce re-runs
  → canonical registry changes → derive re-runs
  → topics with changed topic_membership_hash → render re-runs
  → verify + publish re-run
```

Convergence property (within a cache snapshot): given a cache C, any incremental compile drawing from a subset of C produces the same canonical topics as a full compile over C — identity stable via slug continuity for topics and via cache-returned outputs for everything else. Convergence across independent LLM passes (i.e., a full recompile with C cleared) is not claimed: extract/map/reduce/render are stochastic surfaces, and a fresh pass

is a fresh recording. The cache is therefore both optimization (skip the LLM call) and semantics (define the authoritative recording of this corpus state's LLM outputs).

Partition stability: k-means centroids shift when new ideas are added. Chunk composition near cluster boundaries is unstable, causing some map cache misses. Stable corpus regions form stable chunks and cache-hit across compiles. This is acceptable because re-mapping unchanged chunks is cheap.

Stochastic surfaces and call counts at 10K-doc scale

step	LLM calls	model
extract	~10,000 (1/doc)	DeepSeek v3.2 / V4
map (inside abstract)	~30 (1/chunk)	DeepSeek v3.2 / V4
reduce (inside abstract)	1	Qwen 3.6 Plus (primary)
render	~200 (1/topic)	Qwen 3.6 Plus or Sonnet 4.6

Everything else mechanical. Estimated cost per 10K-doc compile: $17 budget config, $40 quality config.

Invariants (reaffirmed from six-phase, adjusted where needed)

invariant	status
Raw markdown immutable	preserved
Feedback re-enters as source docs	preserved
Batch-agnostic / convergent compilation	preserved via caching + slug continuity
Sessions immutable	preserved
No backcompat shims during refactor	preserved
Agent operates on articles, not concepts	preserved (articles now = topics)
1 Concept = 1 Article	**replaced** by 1 Topic = 1 Article, with many-to-many ideas
Phase 2 clustering is the only place that makes namespace decisions	**replaced** by reduce step is the only place that makes article-inventory decisions
Identity persistence via slug continuity	preserved, moved from concept to topic
No structural mutations from lint	preserved (lint remains detection-only)

What changes from current six-phase

current state	new state
`concepts` table + `concept_repository.py` + clustering in `distiller.py`	retired
6 phases (extract → distill → render → crosslink → index → lint)	7 phases (ingest → extract → abstract → derive → render → verify → publish); lint unchanged as on-demand

`kind` at article level	gone (idea-level only)
`kind` taxonomy hardcoded	per-brain config
Fuzzy-match crosslinker	replaced by intentional authored links verified mechanically
`[[wikilink]]` style (if any)	`[title](wiki/<slug>.md)` markdown style throughout
Per-concept article	per-topic article; ideas are many-to-many
`concept.member_idea_ids`	`topic_membership` derived table
Slug continuity on concepts	slug continuity on topics
Archive flow on concepts (M7)	archive flow on topics
Chunks of rendered articles only	chunks of both raw docs (phase 0) and rendered articles (phase 4)

Points of uncertainty to validate empirically

1. **Qwen 3.6 Plus reduce quality** vs. Sonnet 4.6 vs. Opus 4.7 on merge decisions.
2. **Pre-merge threshold tuning** (cosine > 0.95 may be too strict or too loose).
3. **Map's exhaustive-vs-curated assignment behavior** — what % of ideas end up uncovered.
4. **Partition chunk size sweet spot** — 100K tokens is a guess; may want 50K or 150K.
5. **Token trimming of ideas for map** — does dropping anchors from map's prompt cost thematic judgment, or not.
6. **Whether `link_targets` should be mechanically seeded** (top-K by shared ideas) before reduce, or left fully to the LLM.
7. **Hierarchical reduce threshold** — at what local topic count aggressive pre-merge stops being sufficient.
8. **Brain-level kind taxonomy defaults** — what's a good default set that covers most domains.